

Substitution Method Analysis

Solving the recurrence $T(n) = T(n-1) + n$

NAME

Sai Bhargav

REGISTER NO.

192411049

SUBJECT

Design and Analysis of Algorithms

Problem Definition

What are we analyzing, and with which technique?

The Recurrence

Many recursive algorithms (e.g. loops that shrink by one element per call, or an insertion into a sorted array) reduce a problem of size n to a problem of size $n-1$, plus n units of extra work at the current level.

The Substitution Method

A technique to solve recurrences by:

- Guessing / expanding the recurrence repeatedly
- Substituting the recurrence into itself at each step
- Identifying the resulting pattern in terms of a variable k
- Solving the closed-form summation and verifying it

RECURRENCE RELATION

$$T(n) = T(n-1) + n$$

BASE CASE

$T(1) = 1$ (constant time for the smallest input)

GOAL

Derive a closed-form expression for $T(n)$ and determine its asymptotic (Big-O) growth rate.

Step-by-Step Expansion

Repeatedly substitute the recurrence into itself

Step (k)	Substitution	Resulting Expression
k = 0	Start	$T(n) = T(n-1) + n$
k = 1	$T(n-1) = T(n-2) + (n-1)$	$T(n) = T(n-2) + (n-1) + n$
k = 2	$T(n-2) = T(n-3) + (n-2)$	$T(n) = T(n-3) + (n-2) + (n-1) + n$
k = 3	$T(n-3) = T(n-4) + (n-3)$	$T(n) = T(n-4) + (n-3) + \dots + (n-1) + n$
k (general)	Pattern after k substitutions	$T(n) = T(n-k) + \sum (n-k+1 \dots n)$

Justification: each substitution replaces $T(n-k)$ using the same recurrence, peeling off one term $(n-k+1)$ and reducing the problem size by 1 — the pattern repeats until the base case is reached.

Solving the Summation

Stopping condition, closed form, and formal proof

1 Stopping Condition

Expansion stops when $n-k = 1$, i.e. $k = n-1$, since $T(1)$ is the known base case.

2 Substitute $k = n-1$

$$T(n) = T(1) + [2 + 3 + 4 + \dots + (n-1) + n]$$

3 Apply Arithmetic Series Formula

$$1 + 2 + \dots + n = n(n+1)/2 \rightarrow \text{so } 2+3+\dots+n = n(n+1)/2 - 1$$

4 Closed Form (with $T(1) = 1$)

$$T(n) = 1 + [n(n+1)/2 - 1] = n(n+1)/2$$

Verification: substitute $T(n) = n(n+1)/2$ back into the original recurrence $\rightarrow n(n+1)/2 = (n-1)n/2 + n$, which holds true for all $n \geq 1$.

Final Complexity & Interpretation

CLOSED FORM

$$n(n+1)/2$$

TIME COMPLEXITY

$$\Theta(n^2)$$

GROWTH: n vs. $T(n) \approx n^2$

Efficiency Interpretation

- The total work is the sum of n decreasing terms, giving quadratic growth — the hallmark of nested-loop-style algorithms (e.g. simple sorting, pairwise comparisons).
- Doubling n roughly quadruples the running time, making this class of algorithm inefficient for large inputs compared to $O(n)$ or $O(n \log n)$ alternatives.
- Substitution not only solves the recurrence but formally proves the bound, since each step is verified by direct algebraic reduction.